

ReadRight 2.0 — Milestone 1

Mihir Patel & Mir Patel

Part 5 — Initial Prompt Log

Entry 1 — Understanding Project 1 Architecture (Claude)	
1. Context	We were trying to get a high-level sense of how ngmille/readright organized its state management and repository layers before diving into the code ourselves. We had cloned the repo but main.dart is ~300 lines and we wanted to orient quickly.
2. Prompt Excerpt	"Here is the contents of main.dart and pronunciation_service.dart from a Flutter project. Can you explain how state management is structured and how the speech scoring is triggered? List the key classes and their responsibilities."
3. AI Summary	Claude identified Provider + ChangeNotifierProxyProvider as the state management pattern, flagged that PronunciationAssessor was an interface with two implementations (real and mock), and noted that AssessorFactory.create() always returns the real one, meaning the mock is never actually used. It mapped out the dependency injection chain from main.dart through to the practice screen.
4. Human Evaluation	Mostly accurate. We verified by searching for every call to AssessorFactory.create() in the codebase. Claude was right that it only ever returns the real assessor. The claim about ChangeNotifierProxyProvider was also verified against the actual Provider setup in main.dart. We did not just take Claude's word for it, we cross-checked each class it named.
5. Final Decision	Accepted with one correction: Claude said the mock assessor 'can be enabled via a flag'. We checked and there is no such flag. The mock is simply dead code.
6. Testing / Verification	Searched the repo for 'MockPronunciationAssessor' and it appears only in pronunciation_service.dart itself and in no calling code. Confirmed dead code. Also ran a global search for 'AssessorFactory' to verify it is called in exactly one place.

Entry 2 — Clarifying Azure Integration in Project 2 (Gemini)	
1. Context	We were evaluating Jaredburne47/4150ReadRight and wanted to understand how azure_assessor.dart actually sends audio to Azure — specifically whether it was building the WAV header correctly or just

	passing raw bytes. This mattered for judging how robust the speech pillar actually is.
2. Prompt Excerpt	"Here is azure_assessor.dart. Does this code correctly construct a WAV header before sending to Azure Pronunciation Assessment? What could go wrong with the audio format?"
3. AI Summary	Gemini confirmed that the code manually builds a PCM WAV header (RIFF chunk, fmt subchunk, data subchunk) with the correct values for 16kHz mono 16-bit PCM, which is what Azure expects. It flagged that the Content-Type header is sent as 'audio/wav; codecs=audio/pcm; samplerate=16000' and noted this is more explicit than just 'audio/wav', reducing the chance of Azure misinterpreting the format.
4. Human Evaluation	We verified Gemini's reading by manually checking the byte-construction block in azure_assessor.dart against the PCM WAV spec (RIFF = bytes 0–3, file size at offset 4, 'WAVE' at 8, etc.). The offsets and values matched. Gemini's summary was accurate. It did not flag the hardcoded 3-second recording duration, which we caught ourselves on a separate read-through.
5. Final Decision	Accepted. Gemini's explanation of the WAV header construction was correct and saved us from having to look up the PCM spec from scratch. We added our own observation about the hardcoded duration to the evaluation since Gemini missed it.
6. Testing / Verification	Compared azure_assessor.dart's byte construction against the PCM WAV specification. Confirmed: 'RIFF' at offset 0, file size at offset 4, 'WAVE' at 8, 'fmt ' at 12, subchunk1Size=16 at 16, AudioFormat=1 (PCM) at 20, NumChannels=1 at 22, SampleRate=16000 at 24 — all correct.

Entry 3 — Comparing Data Persistence Models Across All Three Projects (Claude)	
1. Context	After individually evaluating each repo's data layer, we wanted to articulate the tradeoffs clearly for the Codebase Selection section. Specifically: Firestore (Projects 1 & 2) vs. flat JSON file (Project 3). We had our own notes but used Claude to pressure-test our reasoning.
2. Prompt Excerpt	"We are comparing three Flutter apps. Project 1 uses Firestore with a MockRepository fallback. Project 2 uses Firestore + SharedPreferences. Project 3 uses a single flat JSON file for all users and all attempts. We plan to extend one into a multi-student app with AI story content. Which data architecture is easiest to scale and why? What would break first in the flat-JSON approach?"
3. AI Summary	Claude explained that the flat JSON approach breaks at two points: (1) file size grows linearly with students and attempts, making read/write times grow with it; (2) the whole file is rewritten on every save, meaning a crash mid-write can corrupt all data. It recommended Firestore as the better foundation for multi-student, multi-feature growth and noted that

	SharedPreferences is appropriate for lightweight user-preferences (like accessibility settings) but not for relational data like attempt records.
4. Human Evaluation	We agreed with the core argument. We had already flagged the 'full rewrite on every save' issue in Project 3 ourselves from reading the code. Claude's framing helped us articulate it more precisely for the writeup. We were careful not to just copy Claude's wording.
5. Final Decision	Accepted the reasoning; rejected the exact wording. We used Claude's analysis to sharpen our own sentences in the Part 2 selection rationale, but rewrote everything in our own voice. The argument about the flat-JSON corruption risk during an interrupted write is ours. Claude surfaced it, we verified it by reading the save logic in <code>data_persistence_service.dart</code> .
6. Testing / Verification	Read the save method in Project 3's data persistence service. Confirmed: it opens the file, serializes the entire <code>AllUserData</code> object to JSON, and writes the whole string. No atomic write / temp-file-then-rename pattern. Claude's corruption risk assessment was accurate.

Entry 4 — Identifying UX Issues for Young Learners — Project 1 Cupertino Styling (Gemini)	
1. Context	We were writing the User Experience dimension for Project 1 and debating how significant the Cupertino (iOS-style) widget choice really is for a classroom app. We used Gemini to get a second perspective before committing to calling it a real weakness.
2. Prompt Excerpt	"A Flutter literacy app for elementary school children uses Cupertino widgets throughout (CupertinoNavigationBar, CupertinoActivityIndicator, CupertinoButton). Most school devices in the US are Android tablets or Chromebooks. From a UX standpoint, how problematic is this mismatch? Would a child notice or care?"
3. AI Summary	Gemini said the mismatch is a real issue for two reasons: (1) Cupertino components on Android don't respond to the device's Material You dynamic theming, so colors and shapes will feel 'off' on Android; (2) children who have only ever used Android are less likely to intuitively understand iOS-specific conventions like the chevron back-navigation icon, which could create confusion. It also noted that Chromebook Flutter builds render Cupertino widgets but without native touch feedback, making buttons feel unresponsive compared to Material ripple.
4. Human Evaluation	The point about Chromebook touch feedback was new to us and useful. We tested it by checking Flutter's documentation for Cupertino on non-Apple platforms. Gemini's description of the behavior was consistent with what the docs say. The broader point about iOS-vs-Android conventions was our own observation that Gemini confirmed.

5. Final Decision	Accepted. We used this in the UX section for Project 1. We framed it as a 'real UX risk' rather than a dealbreaker, since it can be re-themed. We didn't want to overweight it given Project 1's other strengths.
6. Testing / Verification	Checked Flutter's official documentation on platform-adaptive widgets and the Cupertino library. Confirmed that Cupertino components do not adapt to Material You theming on Android and that touch ripple feedback is Material-specific. Gemini's description was accurate.